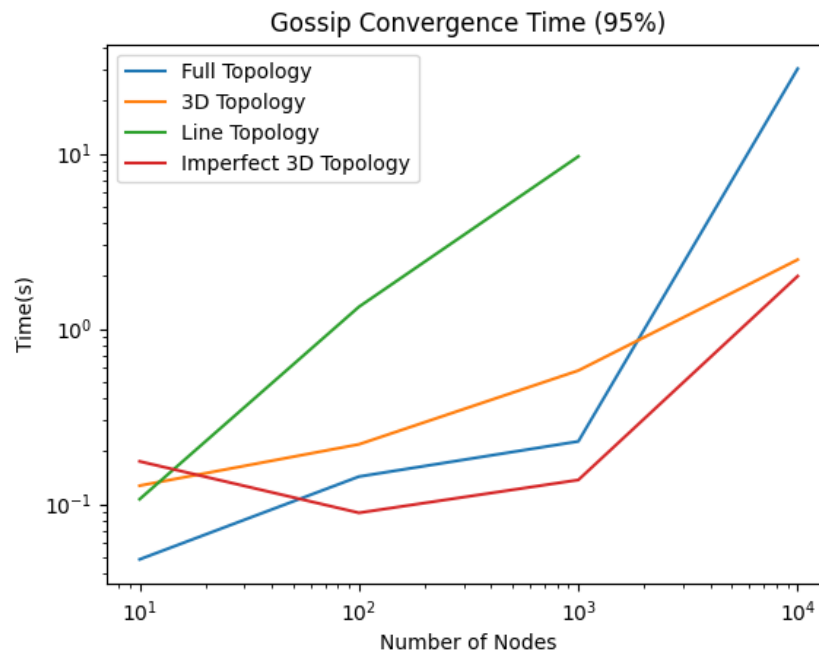


Team Members: Patriel Stapleton and Helen Radomski

Gossip Algorithm

Due to randomness, there is a possibility that some nodes will never become activated or never receive the required 10 messages. This was most apparent with the line topology. To address this, we applied a threshold of 95% convergence to the graph below to keep the threshold the same for all topologies. As such, convergence was reached when 95% of actors had received a message in gossip or maintained a ratio for three rounds in push-sum (in the case of $n=10$, however, a threshold of 90% was used, to allow a leeway of one node). We also chart the performance at 100% convergence later in the section.



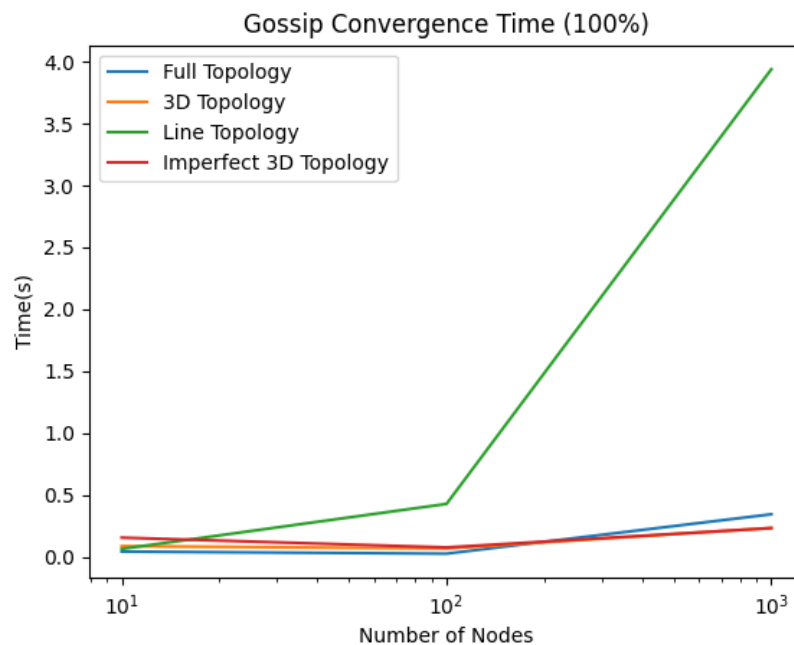
As shown in this graph, line topology performed the worst for the gossip algorithm at most n values and was not able to finish for higher n values. We were able to deduce that this was due to the sparse neighborhood each actor had. Each time an actor received a message, it had a 50% chance of sending that message to a neighbor it had already messaged. This meant actors were more likely to become isolated with no active neighbors. When run with 100% convergence, there was no network size for which the line topology could reliably reach convergence. We expected as much since during a simulation, the line topology only performed perfectly if the first actor chosen was a_1 or a_n , and each subsequent actor would need to transmit to the neighbor they had not received from (excluding a_1 and a_n). Due to randomness and the narrow path to success for this topology, performance was extremely unreliable.

The 3D and imperfect 3D topologies performed best and were not significantly different from each other. The success of this topology can be pictured as a ripple effect, where activity stems from the activation point and spreads in every direction. There are multiple paths between nodes, so they are less likely to become isolated. Each newly activated node opens up multiple new options for

future activations. The imperfect 3D algorithm was slightly faster than the 3D in most cases. We observed that this was because a random actor would be activated somewhere else in the topology. Since our algorithm ensured the random actor would be chosen from outside the immediate neighborhood, this meant that if the random actor was selected during a round, it would trigger activation for a number of actors that would otherwise have to wait for growth to spread node by node.

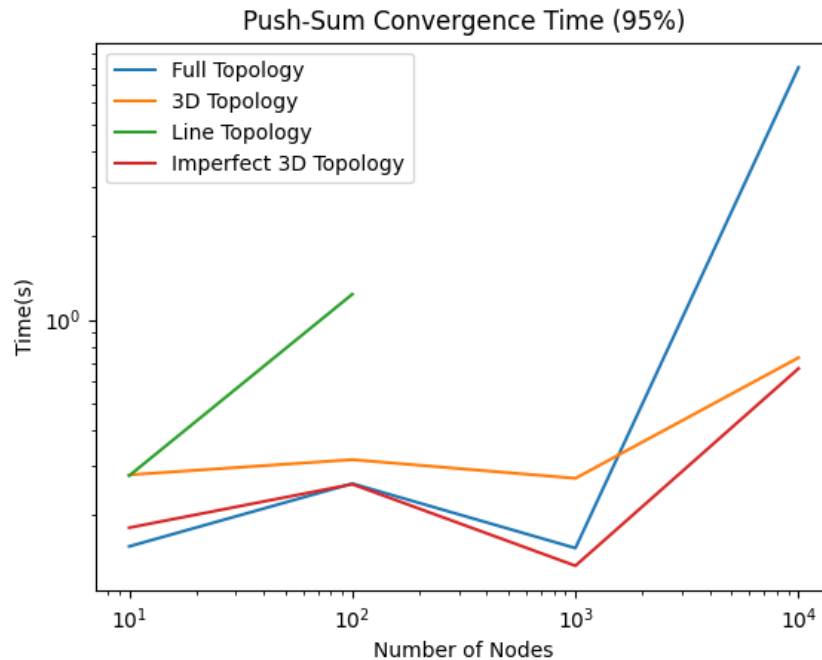
The full topology performed well in small tests but degraded in time performance as the network size grew and N became larger. We ascertained this was due to the shrinking probability of being selected as the neighborhood grew. Each neighbor has $1/(n-1)$ probability of being selected. This performs well for small networks, as the probability is higher that each node will eventually be selected. At higher values of n , this probability degrades because the algorithm only selects one active node at the start; therefore, each inactive node has a very small chance of becoming activated. As a result, growth moves slowly.

The following plot shows the results of the gossip algorithm run with a 100% convergence threshold. Above $n=1,000$, results were highly variable due to the randomness of selection. Below this range, it is too small to detect the difference in performance between 3D, imperfect 3D, and full topologies.



Push-Sum Algorithm

For push sum testing, we also used a convergence threshold of 95% (or 90% for $n=10$). We used values (10, 100, 1_000, 10_000) for full, 3D, and imperfect 3D topologies, but were only able to use (10, 100) for the line topology.



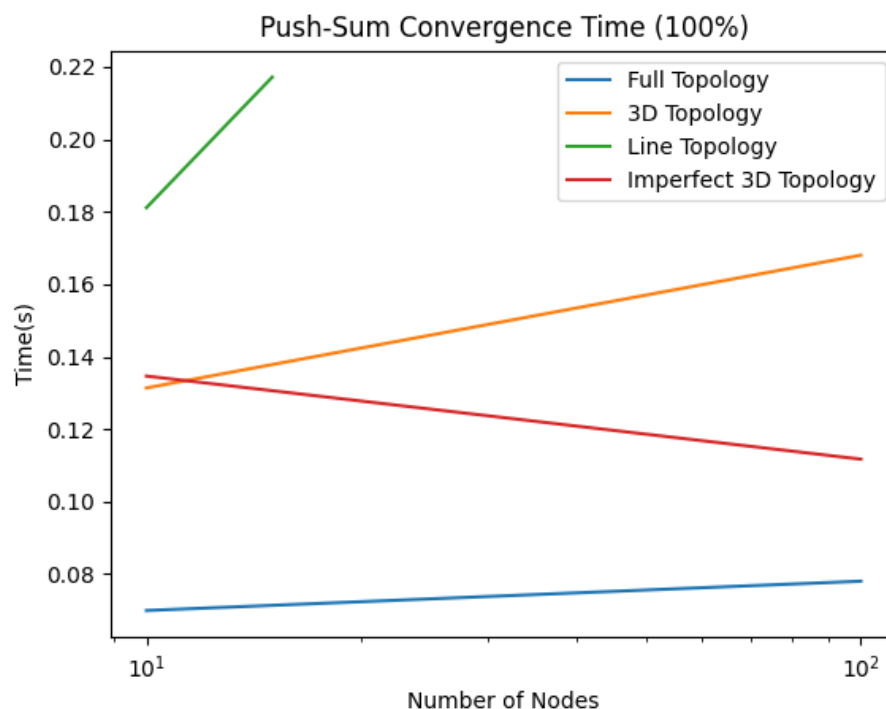
As seen above, the worst-performing topology for the push-sum algorithm was the line topology. This follows the same reasoning as when applied to the gossip algorithm, but in the case of push-sum, the situation is even more dramatic. A node will deactivate itself after just 3 rounds of not receiving any transmissions, making it even more likely that premature deactivation will impede convergence. With just one or two neighbors in a line topology, it becomes more likely that such a situation will happen. All tests run with line topology for values higher than 100 showed good growth initially, but eventually reached a stalling point. We suspect that due to the large network size and the higher probability of a node becoming deactivated, it becomes extremely likely that somewhere along the line, a node becomes inactive too soon, choking its neighbors and stopping progress. We tested this by increasing the number of rounds for which an actor at convergence would still transmit information. With this, we were able to double the network size that the line topology could support before becoming stalled. This suggests that for the push-sum algorithm, when using a sparse neighborhood, the higher the threshold for becoming inactive, the better the performance results.

Our observations for the full topology when executing the gossip algorithm were similar to the push-sum algorithm. As outlined in the graph above, the full topology outperforms both the 3D and imperfect 3D topologies after $n=1,000$. This is due to the changing probabilities. In the full topology, it does not matter if a node becomes inactive early on in the process because its neighbors still have a $1/(n-1)$ chance of becoming activated by another node. Since every node is aware of every other in a fully connected topology, a node becoming inactive does not lead to stalling. However,

we realize this does come at a cost of time. Since inactive nodes are not removed from an actor's neighborhood, they are just as likely to be selected and may take some time to be selected. This is simply one of the tradeoffs of the gossip protocol since no single node manages connections.

Lastly, we have the 3D and imperfect 3D algorithms, which both performed well in push-sum, as expected. Again, the imperfect model can be seen to perform slightly better than the perfect 3D, for the same reason. Overall, these models perform even better than they did under the gossip algorithm. When we lowered the threshold for convergence in the gossip algorithm to 3, we saw a similar increase in time to convergence. As a neighborhood of nodes becomes active, they will continue to stimulate each other, preventing deactivation until growth slows and convergence sets in all at once.

The following plot shows the push-sum performance at the 100% convergence threshold. In this case, performance was only reliable up to $n=100$. For $n=1,000$, the algorithm consistently reached convergence of ~ 99.97 - 99.99 before reaching a stalling point in each topology (excluding line, which performed more poorly).



It is understandable that perfect convergence is harder to achieve under the push-sum algorithm than the gossip algorithm, as nodes may become deactivated after just 3 rounds of inactivity, without having activated all of their neighbors (as opposed to the 10-message threshold in the gossip model). Consider, for example, a node 'a' in the line topology, with two neighbors. One neighbor is active and has activated node 'a'; the other neighbor is not yet active. Each round, the active neighbor has a 50% chance of sending a message to node 'a'. Thus, node 'a' has an expected lifetime of 20 rounds before deactivating ($10 \cdot 1/p$). However, in the push-sum algorithm, the odds of receiving no message for 3 rounds in a row lead to an expected lifetime of 14 rounds ($[1 - (1 -$

$p^3/[p(1-p)^3]$). In both algorithms, the likelihood that node 'a' activates its other neighbor at each round is the same. Therefore, because there is a shorter expected lifetime under the push-sum calculation, it is more likely that node 'a' will deactivate before activating its other neighbor in push-sum than in gossip. If such a situation occurs, the other neighbor has no chance of activation and full convergence will never be achieved. In other topologies, the probabilities become much more complicated, as nodes could have any number of active and inactive neighbors, but performance seems to suggest that push-sum is overall less likely to produce 100% convergence than gossip, especially at higher values of n .